

GitHub Copilot for Enterprise Teams — Participant Lab Guide (Java / Spring Boot)

Synechron Technologies Pvt. Ltd. | 1-Day Training

This guide is yours to keep. Every exercise, every prompt, every reference for the day is here. The morning is mostly demo-driven — you watch.

Contents

- [Before You Arrive](#)
- [Lab 1.1 — Prove the Mental Model](#)
- [Lab 2.1 — Rewrite Three Weak Prompts](#)
- [Lab 2.2 — Try the Techniques](#)
- [Lab 3.1 — Your First `copilot-instructions.md`](#)
- [Lab 3.2 — Comment-Driven Development](#)
- [Lab 4.1 — Built-in Slash Commands with Direction](#)
- [Lab 4.2 — Build Your Own Slash Command](#)
- [Lab 4.3 — Agent Mode for a Multi-File Refactor](#)
- [Lab 6 — Build Sprint](#)
- [Quick Reference Card](#)

Before You Arrive

Software

Tool	Why	Install
Git	Version control	https://git-scm.com/downloads
JDK 21 (LTS)	Runtime for the build sprint	https://adoptium.net (Temurin) or your team's standard
Maven 3.9+	Build tool	https://maven.apache.org/install.html — or just use the Maven wrapper that the scaffolded project generates
IDE	Editor with Java + Copilot support	VS Code with Microsoft's Extension Pack for Java, OR IntelliJ IDEA (Community is fine)
GitHub CLI	gh copilot, auth	https://cli.github.com

GitHub

- Active GitHub account.
- GitHub Copilot **paid seat** assigned and active. Verify at <https://github.com/settings/copilot>.
- GitHub CLI authenticated: run `gh auth login` and follow the browser flow.

IDE Extensions

If using VS Code:

- **GitHub Copilot**
- **GitHub Copilot Chat**
- **Extension Pack for Java** (Microsoft) — bundles Language Support for Java, Debugger, Test Runner, Maven, Project Manager
- **Spring Boot Extension Pack** (VMware / Pivotal)
- **REST Client** (Huachao Mao)

If using IntelliJ IDEA:

- **GitHub Copilot** plugin
- IntelliJ has built-in Maven, Spring Boot, and HTTP client support — no additional extensions needed for those.

Verify You're Ready

In a terminal:

```
git --version
java --version          # Should print Java 21.x
mvn --version           # Should print Apache Maven 3.9.x or later
gh --version
gh auth status
```

All five should succeed. If any fail, contact the trainer before the session.

In your IDE:

- Create a new file `Scratch.java`. Paste:

```
public class Scratch {
    // fibonacci method
}
```

- Place your cursor on a new line inside the class. You should see grey "ghost text" appear with a Copilot suggestion. Press Tab to accept.

If you don't see ghost text — your Copilot extension isn't connected. Check the Copilot icon in the status bar.

Lab 1.1 — Prove the Mental Model

Time: 3 min (during Module 1) **Goal:** See for yourself that the same prompt can produce different output, and that specificity dramatically improves consistency.

Steps

1. Open your IDE. Create a new file `DateParser.java`.
2. Paste:

```
public class DateParser {
```

```
// parse a date
}
```

Place your cursor on the line after the comment. Note what Copilot suggests. Reject (`Escape`).

3. Delete the suggestion. Trigger Copilot again on the same line. Note the suggestion. Different? Reject.

4. Replace the comment with:

```
/**
 * Parses an ISO-8601 date string and returns an Instant.
 * Returns null if the input is invalid. Never throws.
 */
```

Trigger Copilot. Compare to attempts 1 and 2.

What to notice: the third version is more consistent across runs, and almost certainly more correct (probably uses `Instant.parse` inside a try/catch, returns `null` on `DateTimeParseException`). You didn't change the model. You changed the prompt.

Lab 2.1 — Rewrite Three Weak Prompts

Time: 15 min (Module 2) **Goal:** Apply the Prompt Formula on weak prompts and see the output quality jump.

The Formula

ROLE + TASK + CONTEXT + CONSTRAINTS + OUTPUT FORMAT

Weak Prompts

For each, write a strong version using the formula, then run both in Copilot Chat. Note the difference.

Prompt 1: add caching to this method

Set up: paste this method into a Spring `@Service` class. Select it.

```
@Service
public class UserProfileService {

    private final RestClient restClient;

    public UserProfileService(RestClient.Builder builder) {
        this.restClient = builder.baseUrl("https://api.example.com").build();
    }

    public UserProfile getUserProfile(String userId) {
        return restClient.get()
            .uri("/users/{id}", userId)
            .retrieve()
            .body(UserProfile.class);
    }
}
```

```
}  
}
```

Run weak prompt as Inline Chat (`Ctrl+I` / `Cmd+I`):

```
add caching to this method
```

Now write your strong version. Hints — what does Copilot need to know?

- What kind of cache? Spring Cache abstraction (`@Cacheable`)? Caffeine directly? Redis?
- TTL?
- Cache key strategy?
- Invalidation?
- Cache hit/miss behaviour on errors?
- Eviction policy?

Your strong version (write here):

```
[your prompt]
```

Run it. Compare the two outputs.

Prompt 2: write tests

Set up: keep the same `UserProfileService` . Select the `getUserProfile` method.

Weak prompt:

```
write tests
```

Write your strong version. Hints:

- Test framework? (JUnit 5 / TestNG)
- Assertion library? (AssertJ / Hamcrest / plain JUnit)
- Mocking strategy for the `RestClient` ?
- Spring context? (`@WebMvcTest` , `@SpringBootTest` , or plain unit test?)
- Edge cases to cover?
- Test class naming convention?

Your strong version:

```
[your prompt]
```

Prompt 3: fix the bug

Set up: paste this deliberately buggy method into a `@Service` class:

```
/**  
 * Returns the last even number in the list.  
 */  
public Optional<Integer> findLastEven(List<Integer> numbers) {  
    for (int i = 0; i < numbers.size(); i++) {  
        if (numbers.get(i) % 2 == 0) {  
            return Optional.of(numbers.get(i));  
        }  
    }  
}
```

```
        }  
    }  
    return Optional.empty();  
}
```

(Bug: returns the *first* even number, not the *last*.)

Weak prompt:

```
fix the bug
```

Write your strong version. Hints:

- What is the expected behaviour?
- How do you know there's a bug? Reproduce input?
- Constraints on the fix? (Still O(n)? Must not allocate?)
- Style preferences? (Iterate backwards? Use Streams?)

Your strong version:

```
[your prompt]
```

Debrief Questions

- Which weak prompt was most damaging?
 - Which element of the formula made the biggest difference: context, constraints, or output format?
-

Lab 2.2 — Try the Techniques

Time: Self-paced; explore during breaks or after the session. **Goal:** Build hands-on familiarity with each prompting technique using Java examples.

For each technique below, try the example in Copilot Chat and observe what Copilot does.

Zero-Shot

```
Write a Java method `String slugify(String input)` that converts  
a string to a URL-safe slug. Replace spaces and underscores with hyphens,  
lowercase, strip non-alphanumeric except hyphens, collapse multiple hyphens.  
Method should be in a utility class with a private constructor.
```

Few-Shot

```
Convert these snake_case strings to camelCase. Examples:
```

```
snake_case_string -> snakeCaseString  
api_response_data -> apiResponseData  
user_id -> userId
```

```
Now convert: shipping_address_line_2
```

Chain of Thought

Before writing the code, list the edge cases for parsing a CSV row that may contain quoted fields with embedded commas, escaped quotes, and trailing whitespace. Then write the parser method as a Java method returning `List<String>`, handling each case.

Role Prompting

Acting as a senior application security engineer reviewing code for OWASP Top 10 issues, review the following Spring REST controller and identify any security concerns:

[paste a controller]

Micro-Task

Don't ask "build the user module." Break it down:

Write only the `CreateUserRequest` record with Bean Validation annotations. Fields: email (valid email), password (min 8 chars, at least one digit via `@Pattern`), name (1-100 chars). Output only the record, nothing else.

Then a separate prompt:

Write only the controller method that uses the `CreateUserRequest` record above with `@Valid`, calls `UserService.create()`, and returns `ResponseEntity` with status 201 and the created user. Validation failures bubble to the global handler.

Stepwise

Implement a CSV export endpoint in three steps:

Step 1: Define the record for the data row.

Step 2: Write the service method that converts a `List<Row>` to a CSV string, handling quoting and escaping.

Step 3: Write the Spring REST controller method that calls the service and returns `ResponseEntity<Resource>` with the right Content-Type and Content-Disposition headers for download.

Do them in order.

Specification-First

```
/**
 * SPEC:
 *
 * Class: RateLimiter
```

```

* Purpose: Token-bucket rate limiter as a Spring component.
*
* Configuration (from application.yml via @ConfigurationProperties):
*   - maxRequests: number of requests allowed
*   - windowMs: time window in milliseconds
*
* Behaviour:
*   - Track per-key (caller passes a key, e.g. IP or user id) via a
*     ConcurrentHashMap.
*   - tryAcquire(key) returns true if allowed, false if rate-limited.
*   - Sliding window: bucket refills based on elapsed time, not fixed reset.
*   - Thread-safe.
*
* Constraints:
*   - No external dependencies beyond Spring core.
*   - In-memory only (single instance; not Redis-backed).
*   - Implement as a Spring @Component.
*/
// Implement based on the spec above.

```

Cursor-Aware

Place your cursor inside an existing method body where you want a suggestion. Copilot reads forward and backward — that placement matters.

```

public OrderResult processOrder(Order order) {
    validateOrder(order);
    // <- place cursor here, type "// charge the customer via PaymentService", trigger
    Copilot
    notifyWarehouse(order);
    return new OrderResult(order.id(), OrderStatus.PLACED);
}

```

Context Injection via Comments

```

// Note: the `users` table uses `id` (UUID), `email`, `created_at`.
// It has NO `name` column — names are in the separate `profiles` table.
// Use JdbcTemplate to query users joined with their profile name.

```

Output Scaffolding

```

@Service
public class OrderService {

    private final OrderRepository orderRepository;
    private final PaymentGateway paymentGateway;
    private final ApplicationEventPublisher eventPublisher;

    public OrderService(OrderRepository orderRepository,

```

```

        PaymentGateway paymentGateway,
        ApplicationEventPublisher eventPublisher) {
    this.orderRepository = orderRepository;
    this.paymentGateway = paymentGateway;
    this.eventPublisher = eventPublisher;
}

@Transactional
public Order placeOrder(PlaceOrderRequest request) {
    // 1. Validate request

    // 2. Reserve inventory

    // 3. Charge payment

    // 4. Persist the order

    // 5. Publish OrderPlacedEvent

    // 6. Return the order

}
}

```

Constraint-Based

Write a Java method to deeply copy a domain object graph.

CONSTRAINTS:

- No external libraries (no Apache Commons, no Jackson).
- Use a generic method `<T> T deepCopy(T source)`.
- Must handle: records (Java 21), collections (List, Set, Map), nested objects, Instant.
- Must NOT handle: classes with non-default constructors (return as-is).
- No reflection magic – use Cloneable or serialization-based copying.
- Document the limitations in Javadoc.

Positive + Negative Framing

Refactor this method to be more readable.

PREFER:

- Early returns over nested if/else.
- Named constants over magic numbers.
- Streams over imperative loops where clarity improves.
- Records over POJOs for data carriers.

AVOID:

- Mutation of input arguments.
- Renaming public method or class names that other files depend on.

- Adding new dependencies.
- Field injection.

Iterative Refinement

Round 1:

Write a Java method to merge two sorted lists of integers, preserving sort order.

Round 2 (same chat):

Now make it generic – accept a `Comparator<T>` so it works for any type, not just integers. Both input lists are `List<T>`.

Round 3:

Now add Javadoc with `@param`, `@return`, and an example block. Also handle null inputs by treating them as empty lists.

Prompt Chaining

Round 1:

Define the Java record `OrderEvent`. Fields: `orderId` (UUID), `status` (one of: `PLACED`, `PAID`, `SHIPPED`, `DELIVERED`, `CANCELLED` as an enum), `occurredAt` (Instant), `payload` (Map<String, Object>).

Round 2 (use Round 1's output as the foundation):

Using the `OrderEvent` record from above, refactor it into a sealed interface with one permitted record per status value, each with strongly-typed payload fields. So `OrderEvent.Placed` has placement-specific fields, `OrderEvent.Paid` has payment-specific fields, etc.

Lab 3.1 — Your First `copilot-instructions.md`

Time: Self-paced during Module 3. **Goal:** Write a starter instructions file for your team's Java/Spring standards.

In the Build Sprint repo (or any of your own repos), create `.github/copilot-instructions.md`. Fill in the sections below with **your team's** actual standards.

```
# Project Coding Standards for GitHub Copilot

## Language & Framework
- [Java version? Spring Boot version? Build tool – Maven or Gradle?]
- [Any banned namespaces – javax vs jakarta?]

## Architecture
- [Layered? Hexagonal? CQRS? Where does business logic live?]
- [Package structure conventions?]
- [DI style? Constructor only?]
```

DTOs & Domain

- [Records or classes?]
- [Are JPA entities allowed to leave the service layer?]
- [Mapper strategy – MapStruct, hand-written, ModelMapper?]

Validation & Errors

- [jakarta.validation? Custom validators?]
- [Where does @ControllerAdvice live?]
- [Error response shape?]

Naming

- [Standards for classes, methods, variables, packages?]

Testing

- [Framework? JUnit 5? AssertJ?]
- [Use of @SpringBootTest vs @WebMvcTest vs plain unit tests?]
- [Mockito conventions?]

Logging

- [SLF4J? Lombok @Slf4j?]
- [What never to log?]

Transactions

- [@Transactional placement and conventions?]

What NOT to do

- [Field injection? Common anti-patterns?]

Save it. Commit it. Watch your team's Copilot output improve over the next week.

Lab 3.2 — Comment-Driven Development

Time: 5 min during Module 3. **Goal:** Feel the difference between a one-line comment and a spec block.

Create `PriceCalculator.java` (a Spring `@Component`). Try each style in order.

Style A — One line

```
@Component
public class PriceCalculator {

    // Calculate VAT
}
```

Trigger Copilot inside the class. What does it suggest?

Style B — Spec block

```

@Component
public class PriceCalculator {

    /**
     * Calculate VAT (Value Added Tax) for a given amount.
     *
     * @param amount Pre-tax amount in paise (smallest currency unit).
     * @param vatRate VAT rate as a decimal (e.g., 0.18 for 18%).
     * @return The VAT amount, rounded to the nearest paisa.
     *
     * Edge cases:
     * - Negative amounts: throw IllegalArgumentException.
     * - Zero amount: return 0.
     * - vatRate outside [0, 1]: throw IllegalArgumentException.
     */
}

```

Trigger Copilot.

Style C — Spec + scaffold

```

@Component
public class PriceCalculator {

    /** ...same spec as above... */
    public long calculateVat(long amount, double vatRate) {
        // Validate inputs

        // Calculate raw VAT

        // Round to nearest paisa

    }
}

```

At each `//` comment, let Copilot fill the block.

Observation: the spec + scaffold pattern gives you the most predictable, reviewable output.

Lab 4.1 — Built-in Slash Commands with Direction

Time: 10 min during Module 4. **Goal:** Stop using slash commands as fire-and-forget. Direct them.

Use any method from the Build Sprint (or paste one from the morning).

For each, try the bare version first, then the directed version. Compare.

/explain

Bare:

`/explain`

Directed:

`/explain in 3 bullet points, focused only on the side effects, any I/O, and any transactional behaviour. Skip the obvious. Assume I'm a senior Spring engineer.`

`/fix`

Bare:

`/fix`

Directed:

`/fix the bug, but first explain in one sentence what the bug is, then show the corrected code. Do not refactor anything beyond the bug. Preserve the existing method signature.`

`/tests`

Bare:

`/tests`

Directed:

`/tests using JUnit 5 with @ExtendWith(MockitoExtension.class), AssertJ for assertions. Cover:`

- `- Happy path with valid input`
- `- Empty list / null input`
- `- Single-element input`
- `- Input containing one invalid item among many`

`Use @Mock for dependencies and @InjectMocks for the class under test. Use @DisplayName for each test. Method names follow the should_<behaviour>_when_<condition> pattern.`

`/doc`

Bare:

`/doc`

Directed:

`/doc include @param (with semantic meaning, not just the type), @return, @throws for any thrown exception, and a {@code ...} example block. Keep the summary line under 80 characters. Use British English spelling.`

Lab 4.2 — Build Your Own Slash Command

Time: 15 min (during Module 4) or self-paced. **Goal:** Author a prompt file that becomes a reusable slash command for your team.

Setup

In a workspace, create `.github/prompts/`. Inside, create a `.prompt.md` file. The file name (minus extension) becomes the slash command name. Example: `review-security.prompt.md` becomes `/review-security`.

Template

```
---
description: One-line description shown in the command picker.
mode: ask
---

Acting as [role].

[Your instructions to Copilot]

For each [unit of work]:
- [What to produce]
- [In what format]
- [Any constraints]

Selected code:

${selection}

Additional context: ${input}
```

Variables Available

- `${selection}` — the code the user has selected when invoking the command.
- `${input}` — anything the user typed after the command, e.g. `/review-security pay particular attention to authentication`.
- You can also reference files: `#file:path/to/file.java` in the prompt body.

Ideas to Build (Pick One)

1. `/synechron-javadoc` — Add Javadoc following Synechron's documentation standard (you define it).
2. `/log-instrument` — Add SLF4J logger calls at strategic points (method entry, error paths, important branches).
3. `/spring-controller` — Generate a `@RestController` scaffold following the team's layered architecture.
4. `/jpa-repository` — Generate a Spring Data JPA repository interface from a domain entity, with the team's standard query method naming.
5. `/team-code-review` — Apply your team's code review checklist as a single command (e.g. constructor injection, no field access on entities, transaction boundaries, log statements at the right level).

Build one. Commit `.github/prompts/your-command.prompt.md` to the repo. Test by selecting code and typing `/your-command` in Copilot Chat.

Lab 4.3 — Agent Mode for a Multi-File Refactor

Time: 10 min during Module 4. **Goal:** Use Agent mode for something basic chat can't do well — a coordinated, multi-file change.

Setup

In the build sprint repo (or any small Spring project you have), pick a refactor that touches 3+ files. Example: rename a service, extract a constant, replace a logger.

Prompt

This lab has two parts. **Try both** — the contrast is the lesson.

Part A — Agent alone:

In Copilot Chat, switch the mode dropdown to **Agent**.

Prompt example (adjust for your repo):

```
Refactor: rename the `UserService` interface and `UserServiceImpl` class
to `AccountService` and `AccountServiceImpl` everywhere in the codebase.
Update all imports, all type references, the file names, and any test
classes. Update any Javadoc references. Follow our coding standards in
.github/copilot-instructions.md.
```

Part B — Plan first, then Agent:

Open a fresh Chat session. Switch to **Plan** mode.

Run the same prompt. Notice Copilot now produces an **implementation plan** instead of edits — a summary, the list of files it will touch, and any clarifying questions. This is especially valuable in a Spring codebase where references can hide in non-obvious places (bean names, AOP pointcuts, `application.yml` profiles). Review the plan. Answer any questions. Then click **Start Implementation** to hand off to Agent.

What to Do

- For Part A: approve each step one at a time. **Do not click "approve all"** — you're learning to supervise.
- For Part B: review the plan carefully *before* clicking Start Implementation. Did Copilot list every file? Did it ask about bean names or AOP pointcuts? Catching it in the plan is cheaper than unwinding diffs.
- For each diff Copilot proposes, scan it. Reject any change that's wrong.
- Run your tests after Agent mode finishes (`./mvnw test`). Do they still pass?

Reflection

- What did Agent mode do well in Part A? What did it miss?
 - Did Plan mode in Part B surface anything before execution that you'd have caught only in diff review with Part A?
 - Did either approach update the bean name (`@Service("userService")` becomes `@Service("accountService")` if it was named)?
 - Did any Spring config files (XML, `application.yml` profiles) reference the old name?
-

Lab 6 — Build Sprint

Time: 60 minutes. **Goal:** Build a working Task Management REST API end-to-end, applying everything from the morning.

The full project specification is in `BUILD_SPRINT_PROJECT_JAVA.md`. Read it before you start.

Phase 1: Project Setup (15 min)

Use Copilot Chat in Agent mode to scaffold the project. If you want to review Copilot's plan before any files are created, start in **Plan mode**, read and refine the plan, then click **Start Implementation** to hand off to Agent. Write a strong prompt — use the formula.

You should end Phase 1 with:

- `pom.xml`, `.gitignore`, `application.yml`
- `TaskApiApplication.java` (`@SpringBootApplication` main class)
- `.github/copilot-instructions.md` (your team's standards)
- Package structure: `controller`, `service` (with `impl/`), `repository` (with `impl/`), `domain`, `dto`, `mapper`, `exception`, `config`
- `./mvnw spring-boot:run` starts the server on port 8080
- `GET http://localhost:8080/actuator/health` returns `{"status":"UP"}`

Phase 2: Build the API (25 min)

Implement the five endpoints from the spec (`POST /api/tasks`, `GET /api/tasks`, `GET /api/tasks/{id}`, `PATCH /api/tasks/{id}`, `DELETE /api/tasks/{id}`).

Recommended order:

1. Define the `TaskStatus` and `TaskPriority` enums with `@JsonValue` / `@JsonCreator` for lowercase JSON.
2. Define the `Task` record (domain).
3. Define exceptions: `TaskNotFoundException`, `ValidationException`.
4. Define the `TaskRepository` interface and the `InMemoryTaskRepository` (`@Repository`).
5. Define the service interface and `TaskServiceImpl`.
6. Define DTOs as Java records: `CreateTaskRequest`, `UpdateTaskRequest`, `TaskResponse`, `ListTasksResponse`, `ErrorResponse`. Add `jakarta.validation` annotations.
7. Build the controller layer (`@RestController`, `@Valid`, `ResponseEntity`).
8. Add the global exception handler (`@ControllerAdvice`).

Use Inline Chat for surgical edits. Use Agent mode when adding a new endpoint (controller + service + repository at once).

Phase 3: Wire and Verify (20 min)

- Configure Jackson for snake_case JSON in `application.yml`.
- Verify all `MethodArgumentNotValidException` paths return the team's `ErrorResponse` shape.
- Smoke test using the `.http` file included in the spec — use the REST Client extension or IntelliJ's HTTP client.

Checkpoint

By the end of the build sprint, you should be able to:

```
curl -X POST http://localhost:8080/api/tasks \
  -H "Content-Type: application/json" \
  -d '{"title": "Test task", "description": "A test", "status": "todo"}'

curl http://localhost:8080/api/tasks
```

And both should work, with snake_case fields in the responses.

Quick Reference Card

Keyboard Shortcuts (VS Code)

Action	macOS	Windows / Linux
Accept suggestion	Tab	Tab
Reject suggestion	Esc	Esc
Cycle suggestions	Option+] / Option+[	Alt+] / Alt+[
Open suggestions panel	Cmd+Enter	Ctrl+Enter
Inline Chat	Cmd+I	Ctrl+I
Open Copilot Chat	Ctrl+Cmd+I	Ctrl+Alt+I

Keyboard Shortcuts (IntelliJ IDEA)

Action	macOS	Windows / Linux
Accept suggestion	Tab	Tab
Reject suggestion	Esc	Esc
Cycle suggestions	Option+] / Option+[	Alt+] / Alt+[
Open Copilot Chat	Cmd+Shift+C	Ctrl+Shift+C

The Prompt Formula

ROLE + TASK + CONTEXT + CONSTRAINTS + OUTPUT FORMAT

Built-in Slash Commands

Command	Use For
/explain	Understand selected code
/fix	Fix a bug in selected code
/tests	Generate tests for selected code
/doc	Add Javadoc to selected code

/help	List all available commands
/clear	Start a fresh chat

Chat Context Directives

Syntax	Purpose
#file:path/to/file.java	Pin a specific file as context
#selection	Reference the current editor selection
@workspace	Search the whole workspace
@terminal	Reference recent terminal output
@github	Search GitHub (issues, PRs, repos)

Chat Modes

Mode	What It Does
Ask	Answers questions, suggests code, never edits files
Plan	Generates an implementation plan for your review — summary, file-by-file step breakdown, clarifying questions. No code changes until you hand off to Agent
Agent	Plans + edits + runs commands autonomously across multiple files, you approve each step

Edit mode (from earlier versions of Copilot) was folded into Agent mode. The three modes above are what you'll see in the mode dropdown today.

When to Use Which

Task	Tool
1-line completion	Just type, hit Tab
Refactor in one file	Inline Chat (Cmd+I)
Multi-turn explore	Copilot Chat (Ask mode)
Coordinated multi-file change	Copilot Chat — Plan mode → review → Agent mode
Repeated team workflow	Custom slash command
Domain knowledge across repos	Custom Copilot Extension
Terminal task	gh copilot suggest / Copilot CLI

Java-Specific Pitfalls to Watch For

Symptom	Likely Cause	Fix
---------	--------------	-----

@Autowired field injection in Copilot output	Older Spring training data	Add explicit rule in copilot-instructions.md; start a fresh Chat session
javax.validation.* imports	Pre-Spring-Boot-3 patterns	Same — rule + new Chat session
Generated tests use JUnit 4 (@RunWith)	Same	Same
@Transactional on a controller method	Same	Same
JPA entity exposed directly in HTTP response	Layering violation	Reinforce DTO + mapper rule
Enum values rejected from query params	@JsonCreator not registering for query binding	Add a Converter<String, TaskStatus> bean

When Copilot Goes Quiet — Quick Fixes

1. Click the Copilot status bar icon → check state.
2. Developer: Reload Window from the Command Palette (VS Code) or File → Invalidate Caches (IntelliJ).
3. GitHub Copilot: Sign Out then sign in again.
4. Check <https://www.githubstatus.com>.
5. If a specific file: check if it's in your org's exclusion list (Copilot icon will show "disabled for this file").

End of Participant Lab Guide (Java).

Keep this file. Reference it Monday. The Prompt Formula on the back of your monitor for a week is the cheapest productivity intervention you can make.